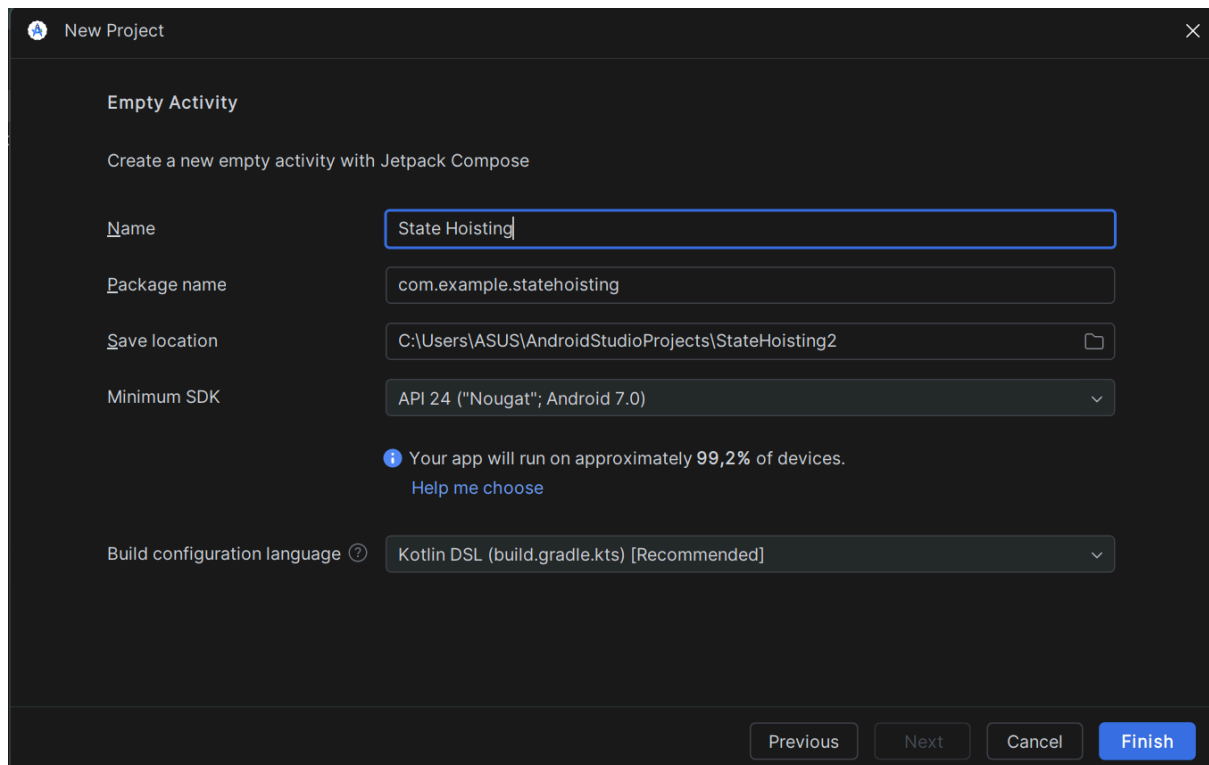


Nama: Kania Khalifa Kurniadi

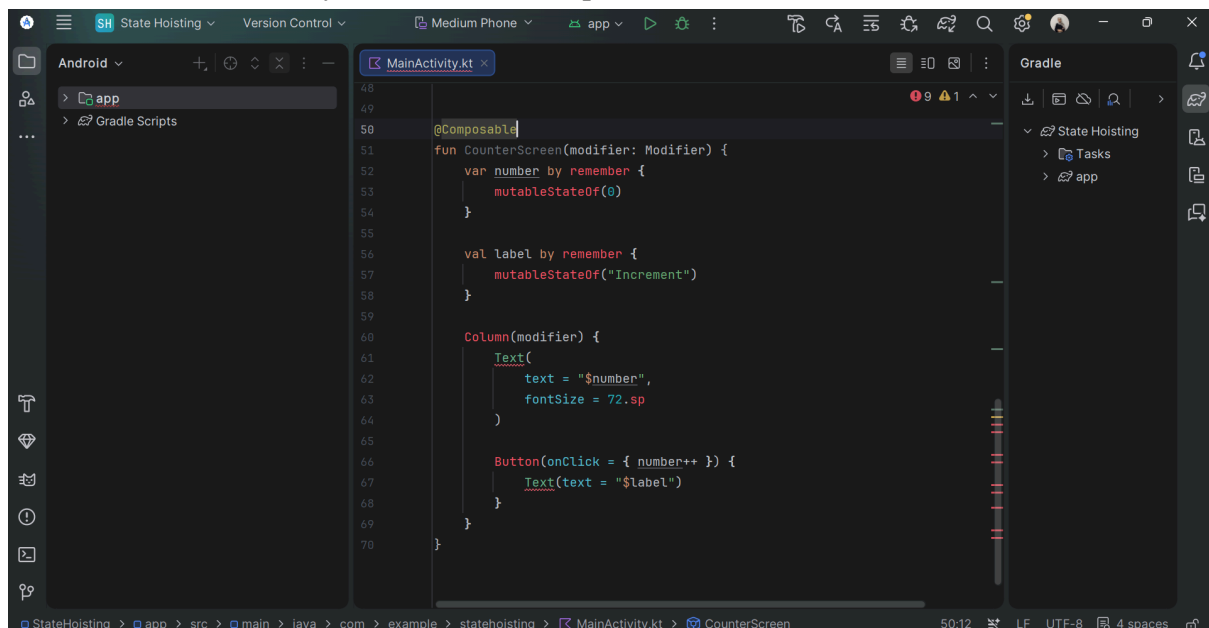
NIM: 245150407111042

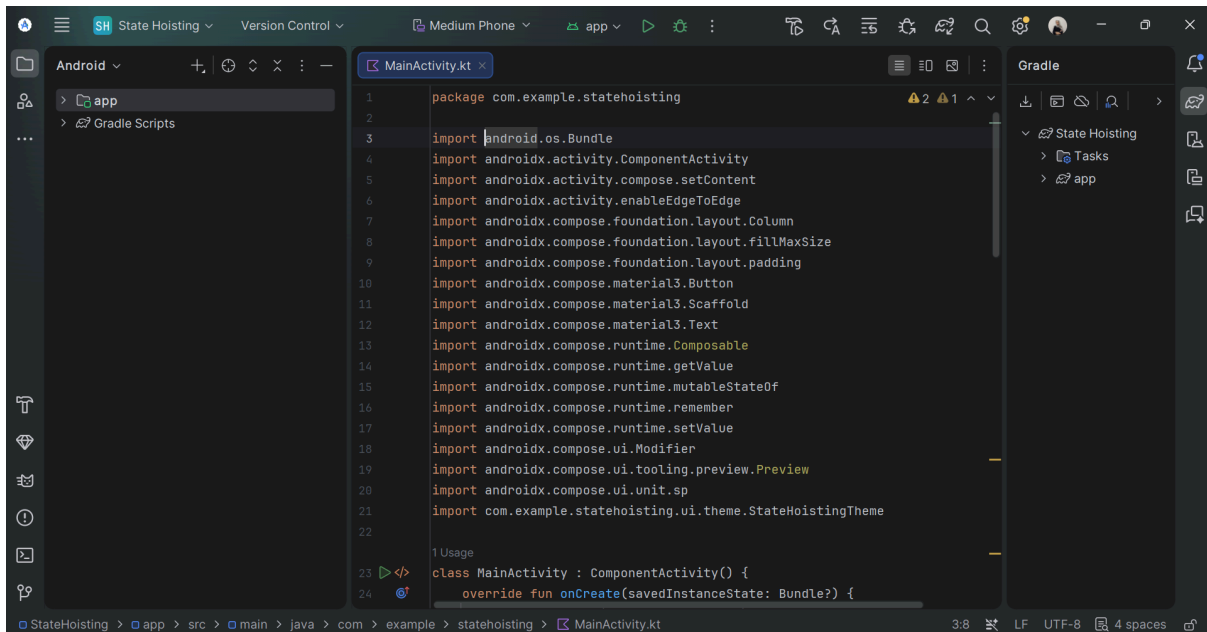
State Hoisting

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.

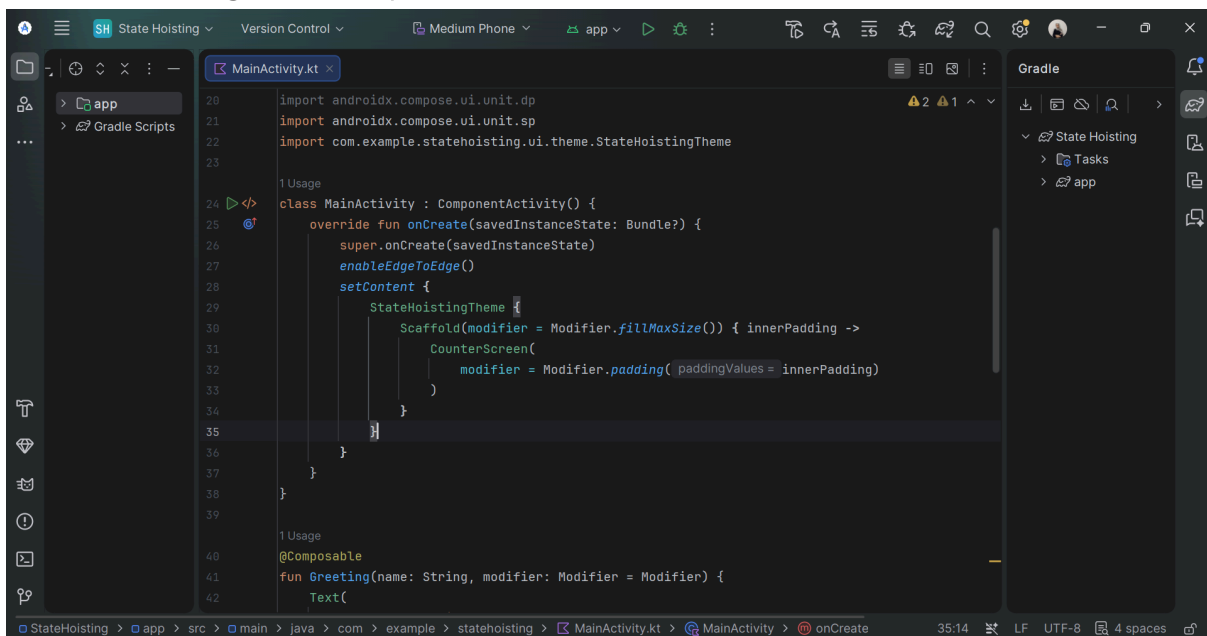


2. ubah MainActivity dan tambahkan import



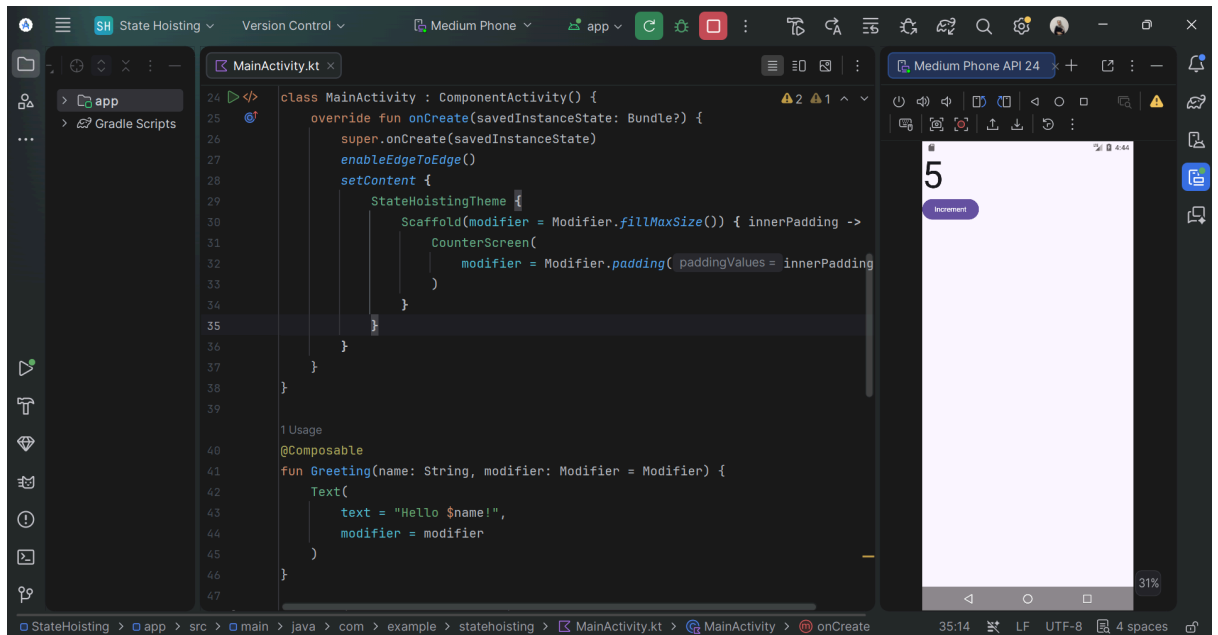


3. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah bagian kode program berikut di dalam fungsi onCreate()



AKTIVITAS PERCOBAAN

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Setiap kali tombol "Increment" diklik, angka yang ditampilkan pada layar akan bertambah 1, dari 0 menjadi 1, lalu 2, 3, dan seterusnya yang menunjukkan bahwa tombol tersebut berfungsi untuk menambahkan nilai angka setiap kali pengguna menekan

b. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena pada bagian Button terdapat fungsi onClick yang menjalankan perintah number++. Perintah tersebut digunakan untuk menambahkan nilai variabel number sebanyak 1 setiap kali tombol diklik. Variabel number juga dideklarasikan sebagai state menggunakan remember dan mutableStateOf, sehingga ketika nilainya berubah, Jetpack Compose akan memperbarui tampilan yang menggunakan state tersebut.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state yang berperan adalah number, karena variabel ini digunakan pada Text melalui text = "\$number". Ketika nilainya berubah, angka yang ditampilkan pada layar juga ikut berubah

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state yang berperan adalah label, karena variabel ini digunakan untuk mengisi teks pada tombol melalui Text(text = "\$label"). Nilai label saat ini adalah "Increment", sehingga tulisan tersebut yang muncul pada tombol

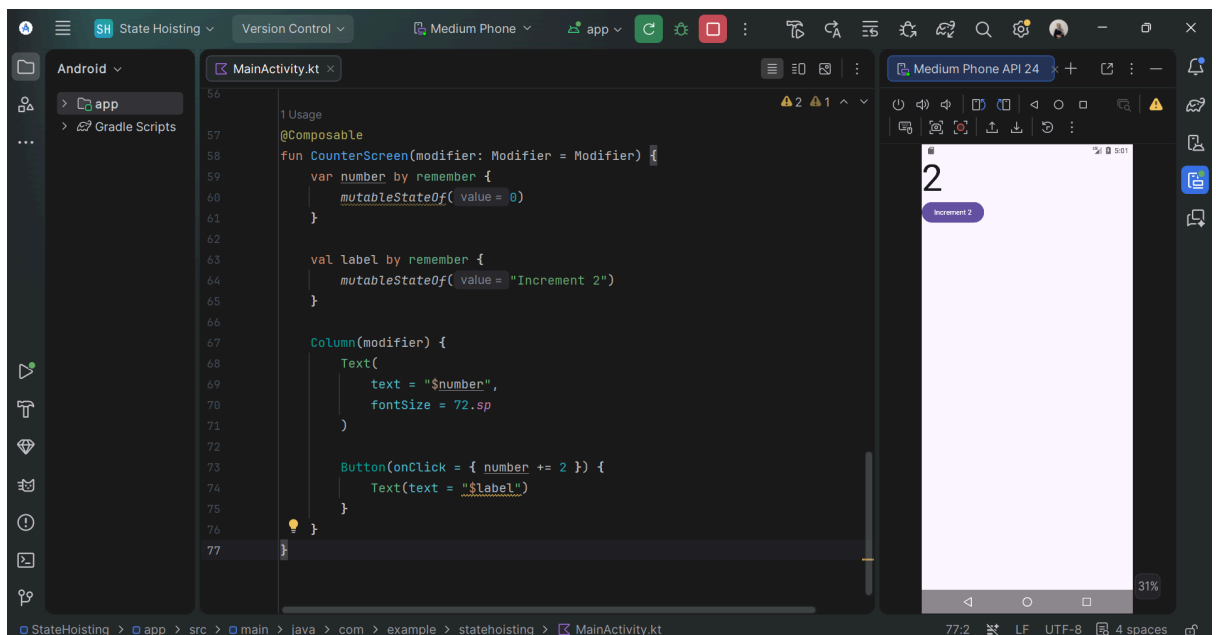
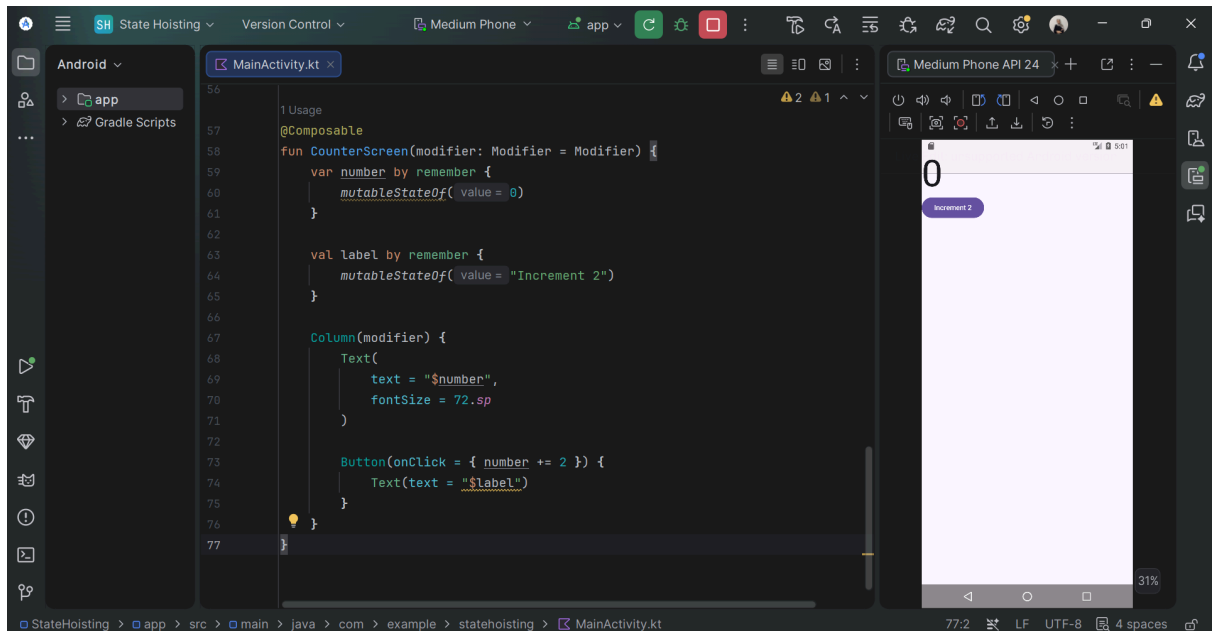
5. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan dilakukan pada fungsi CounterScreen, yaitu dengan mengubah label tombol menjadi "Increment 2" dan mengganti perintah number++ menjadi number += 2. Perubahan

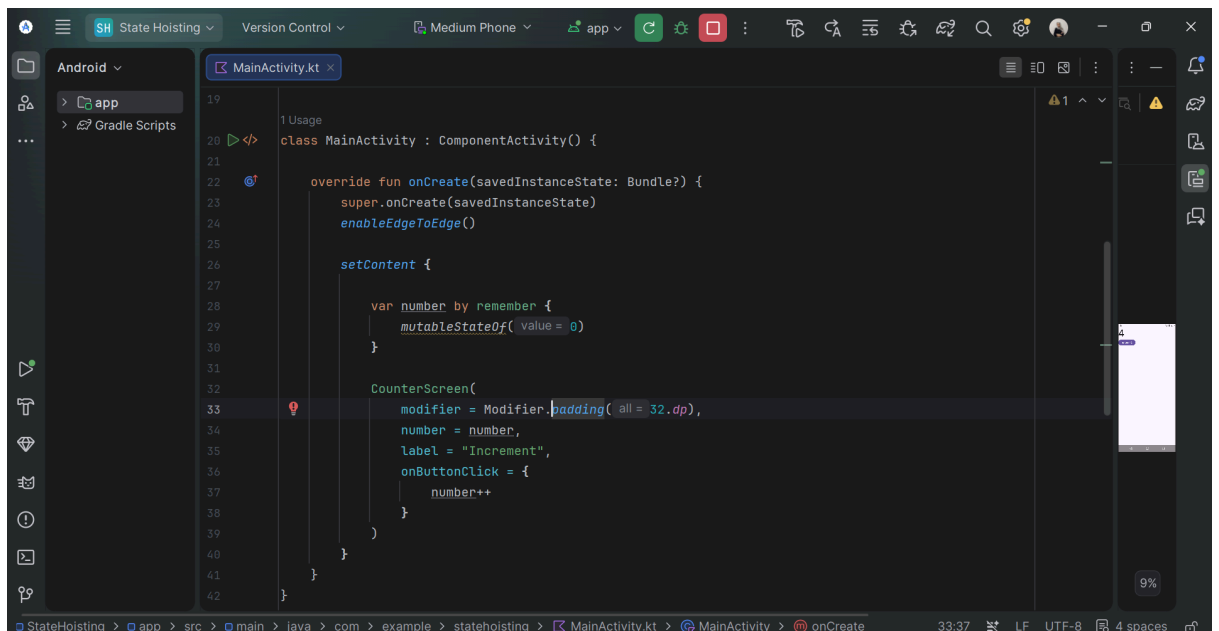
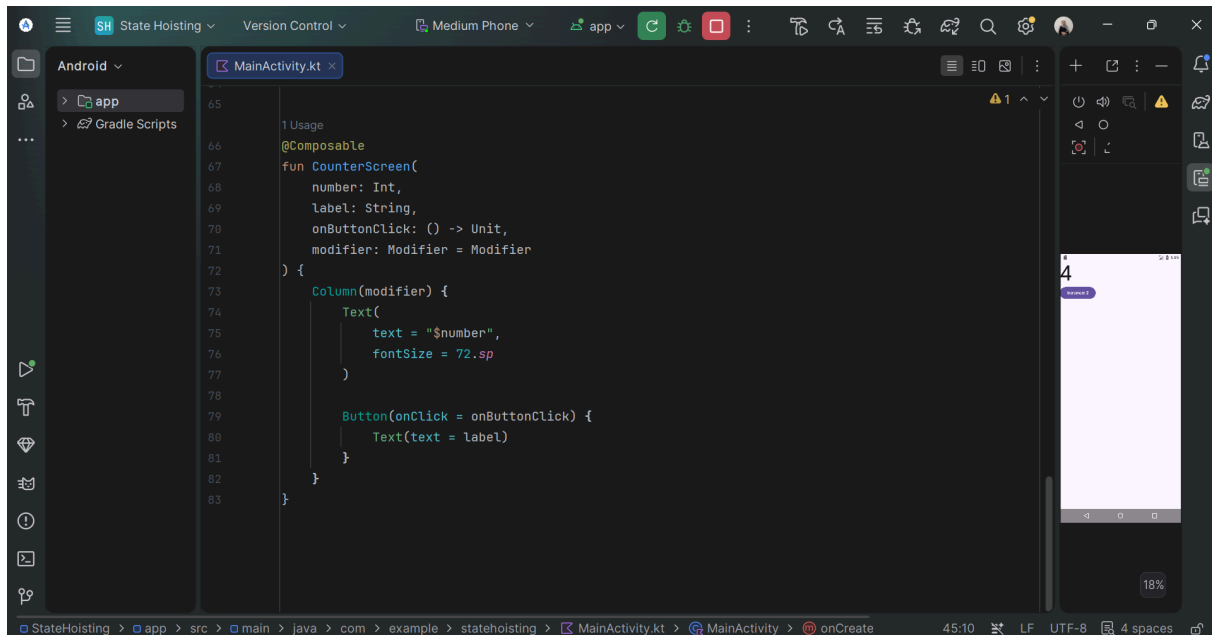
label dilakukan agar tulisan tombol sesuai dengan fungsi barunya, sedangkan perubahan operasi membuat angka bertambah 2 setiap kali tombol diklik.

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

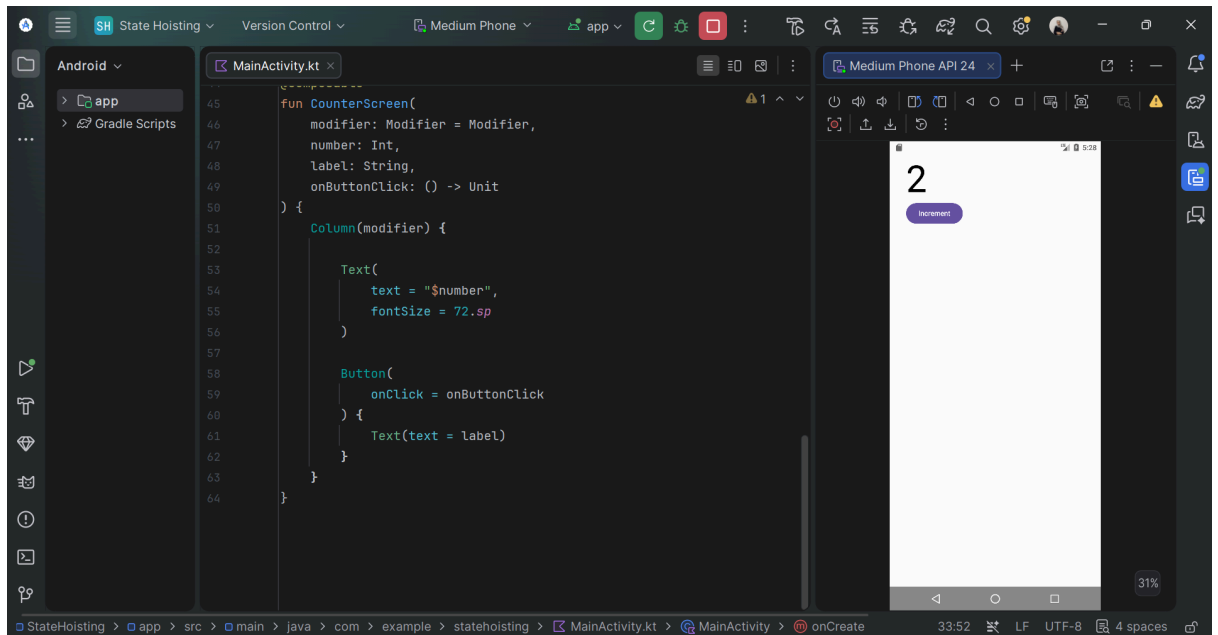


Sebelum tombol diklik, angka yang tampil adalah 0 dan label tombol adalah "Increment 2". Setelah tombol diklik satu kali, angka berubah menjadi 2 karena `number += 2`

- Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen. Ubah signature kode program pada bagian fungsi Composable CounterScreen



7. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.



a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, program tetap berjalan dengan baik dan menampilkan hasil yang sama seperti tahap sebelumnya. Angka tetap bertambah setiap kali tombol "Increment" diklik. Hal ini karena state number tetap dikirimkan ke CounterScreen, sehingga tampilan dapat membaca nilai terbaru meskipun state sudah dipindahkan ke MainActivity.

b. Di bagian kode program manakah saat ini proses increment dilakukan?

increment dilakukan pada: `onButtonClick = { number++ }` yang berada di bagian pemanggilan CounterScreen.

c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State number sekarang dideklarasikan di luar CounterScreen: `var number by remember { mutableStateOf(0) }` Sedangkan CounterScreen hanya menerima nilai: `number: Int` dan event: `onButtonClick: () -> Unit`

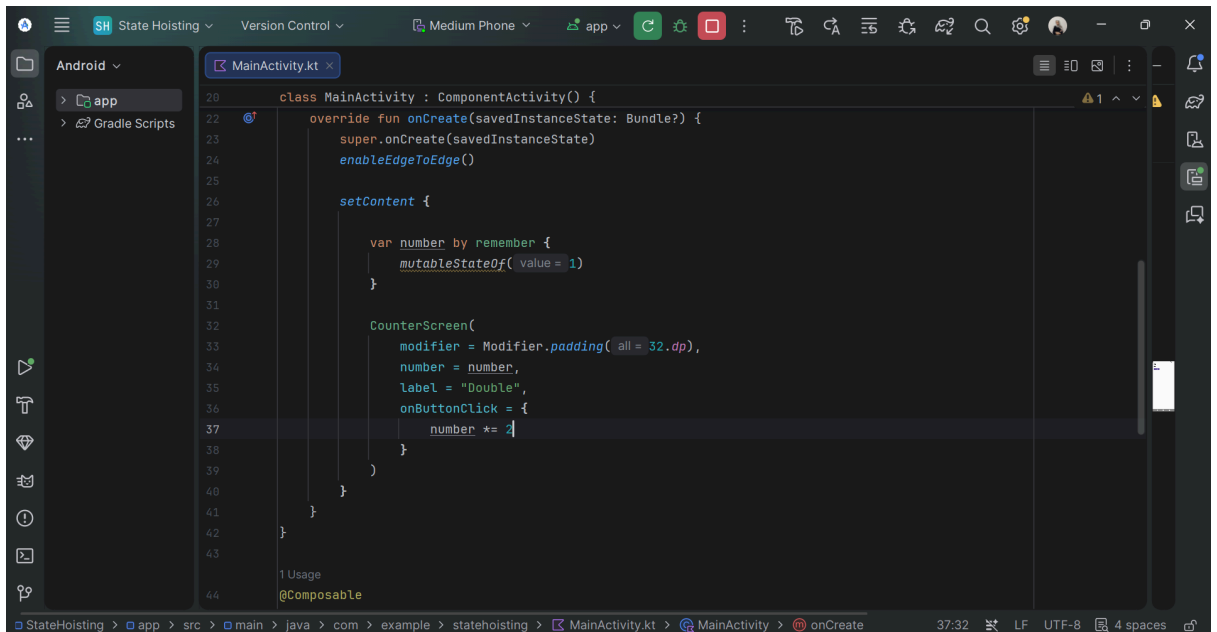
8. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

a. Label tombol "Increment" menjadi "Double"

b. Nilai awal variabel number dari 0 menjadi 1.

c. Ubah fungsi/lambda onButtonClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel number.

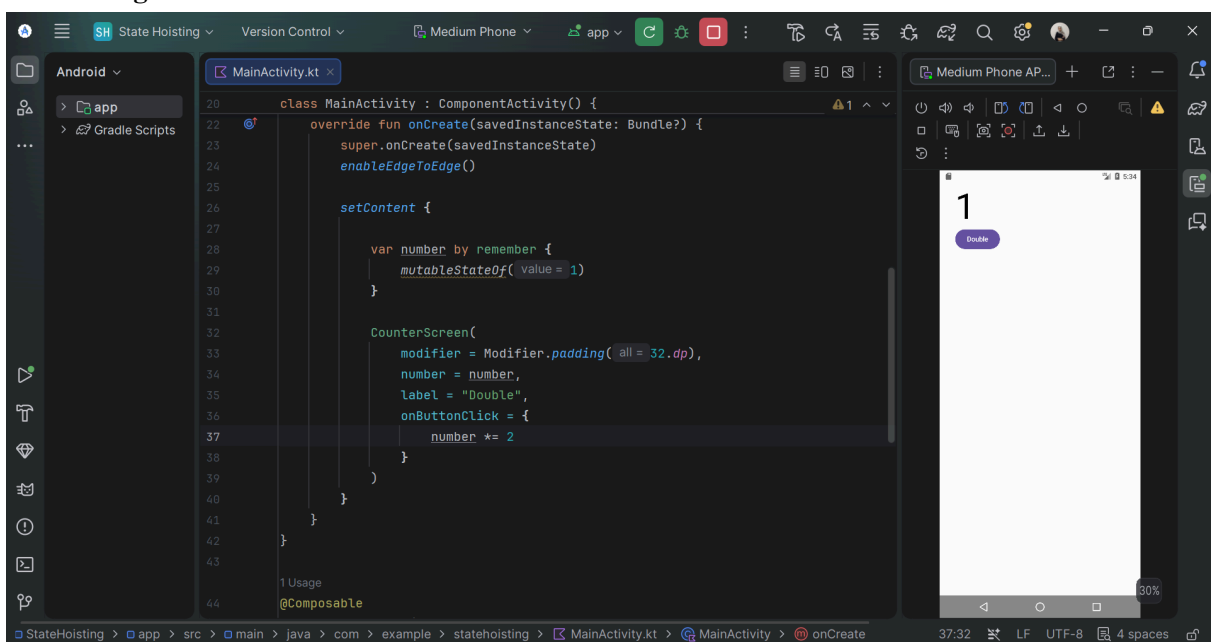
d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.



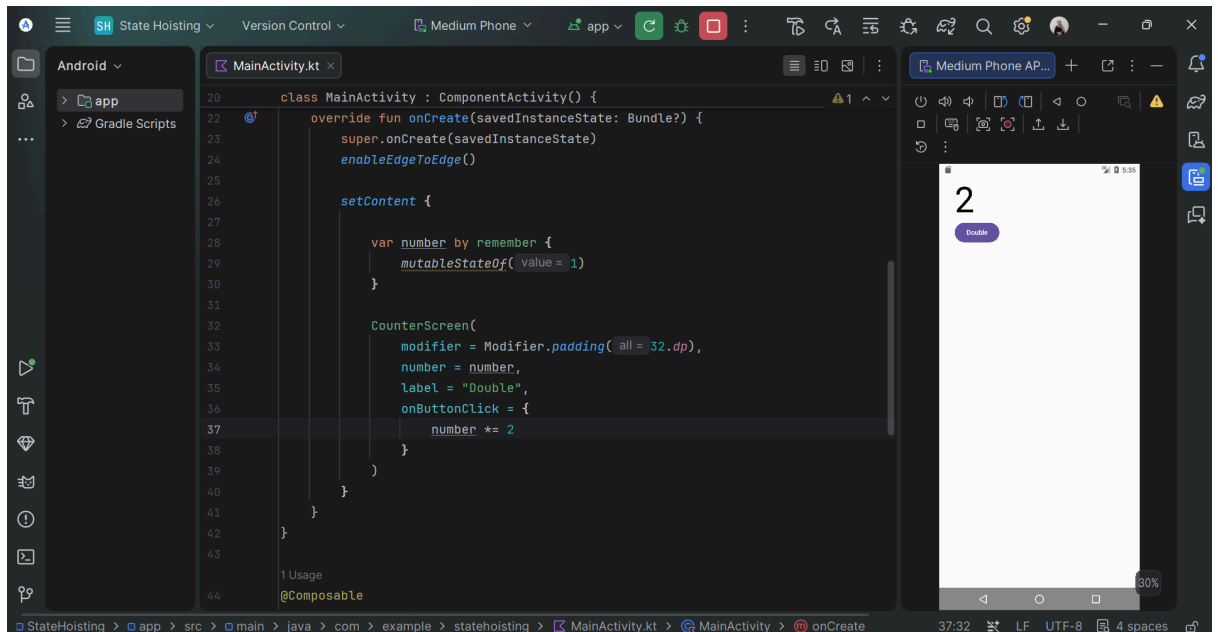
```
20 class MainActivity : ComponentActivity() {
21
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         enableEdgeToEdge()
25
26         setContent {
27
28             var number by remember {
29                 mutableStateOf(value = 1)
30             }
31
32             CounterScreen(
33                 modifier = Modifier.padding(all = 32.dp),
34                 number = number,
35                 label = "Double",
36                 onClick = {
37                     number *= 2
38                 }
39             )
40         }
41     }
42 }
43
44 1 Usage
45 @Composable
```

9. Jalankan program,

- Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



```
20 class MainActivity : ComponentActivity() {
21
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         enableEdgeToEdge()
25
26         setContent {
27
28             var number by remember {
29                 mutableStateOf(value = 1)
30             }
31
32             CounterScreen(
33                 modifier = Modifier.padding(all = 32.dp),
34                 number = number,
35                 label = "Double",
36                 onClick = {
37                     number *= 2
38                 }
39             )
40         }
41     }
42 }
43
44 1 Usage
45 @Composable
```



b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Pada percobaan tahap nomor 7, proses increment dilakukan pada bagian pemanggilan CounterScreen di dalam setContent, tepatnya pada fungsi onClick yang berisi number++.

Pada tahap nomor 8, perubahan logika juga dilakukan pada bagian yang sama, yaitu onClick diubah menjadi number *= 2. Selain itu, pada tahap nomor 8 nilai awal number diubah dari 0 menjadi 1 dan label tombol diubah dari "Increment" menjadi "Double".

Jadi, perubahan pada tahap 7 dan 8 dilakukan pada bagian state dan event yang berada di luar CounterScreen.

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri dari tahap nomor 7 ke tahap nomor 8. Struktur CounterScreen tetap sama, yaitu menerima number, label, dan onClick sebagai parameter, kemudian menggunakan parameter tersebut untuk menampilkan angka dan tombol.

Perubahan hanya dilakukan pada nilai dan fungsi yang dikirimkan ke CounterScreen dari luar. Hal ini menunjukkan bahwa CounterScreen tidak perlu mengetahui bagaimana logika tombol bekerja, karena tugasnya hanya menampilkan UI dan menjalankan event yang diberikan kepadanya.

10. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

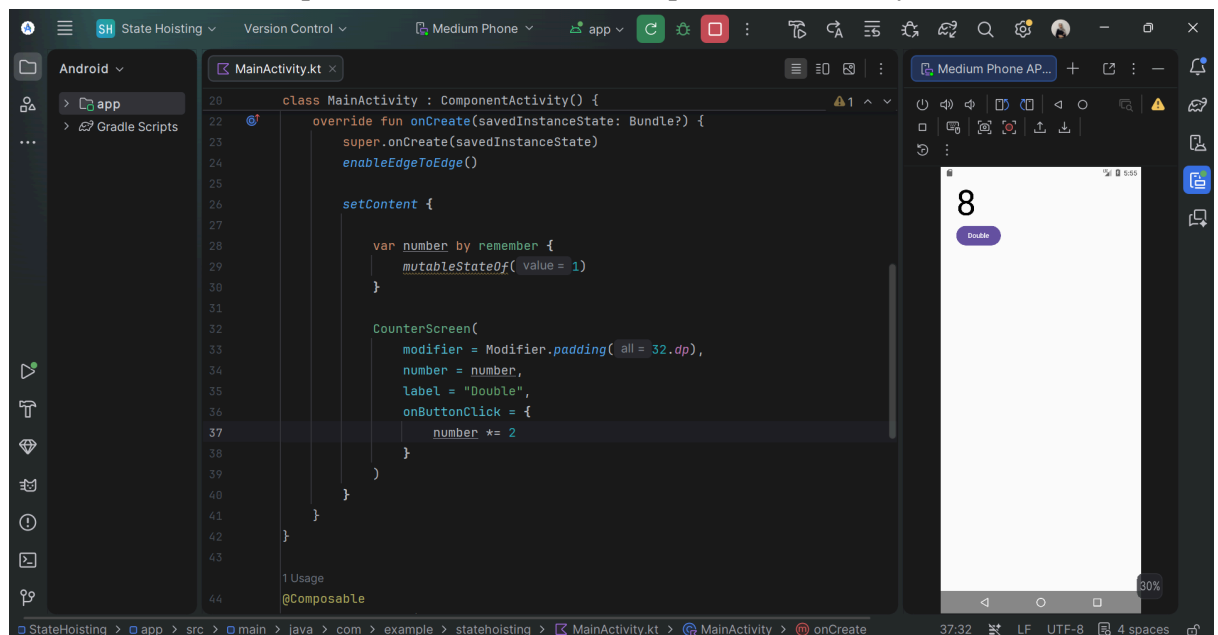
state hoisting membuat perubahan kebutuhan fungsional pada aplikasi menjadi lebih mudah dilakukan tanpa harus mengubah bagian UI secara langsung. Pada percobaan ini, fungsi tombol awalnya adalah menambahkan angka dengan `number++`, kemudian diubah menjadi menggandakan angka dengan `number *= 2`. Meskipun logikanya berubah, fungsi `CounterScreen` tetap sama karena state dan event sudah dipindahkan ke luar `Composable`. Tampilan dan logika program menjadi lebih terpisah sehingga kode lebih fleksibel dan perubahan fungsi dapat dilakukan dengan lebih sederhana.

Perubahan Konfigurasi

1. Untuk percobaan terkait Perubahan Konfigurasi aplikasi Android, gunakan Project Android Studio yang sama dengan kode program percobaan State Hoisting setelah menyelesaikan tahap nomor 6 percobaan tersebut.
2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel `number`).

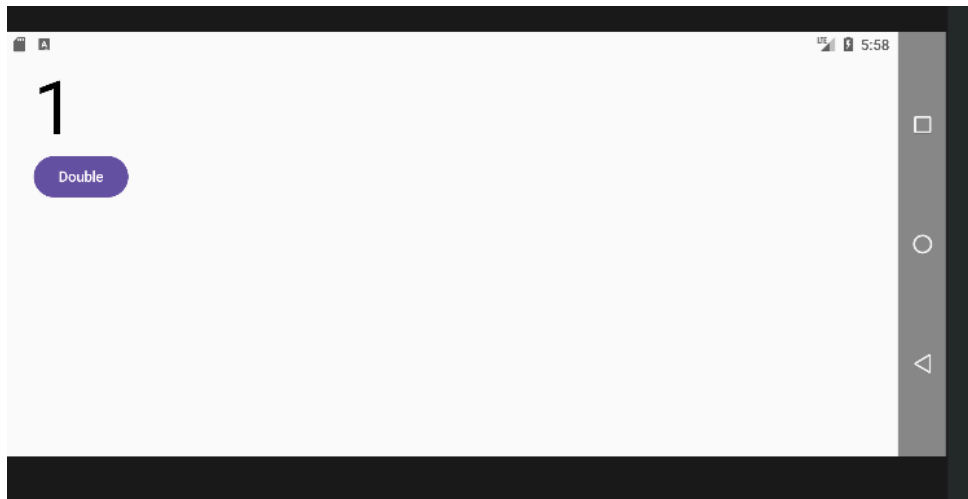
AKTIVITAS PERCOBAAN

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



angka yang ditampilkan 1-2-4-8

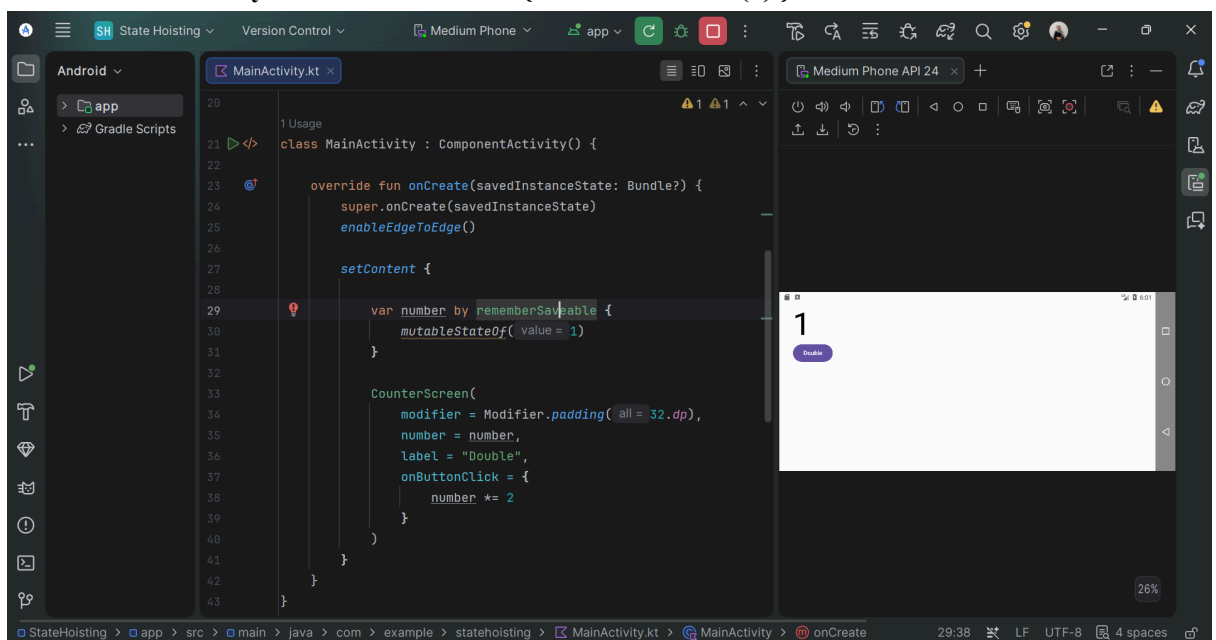
2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



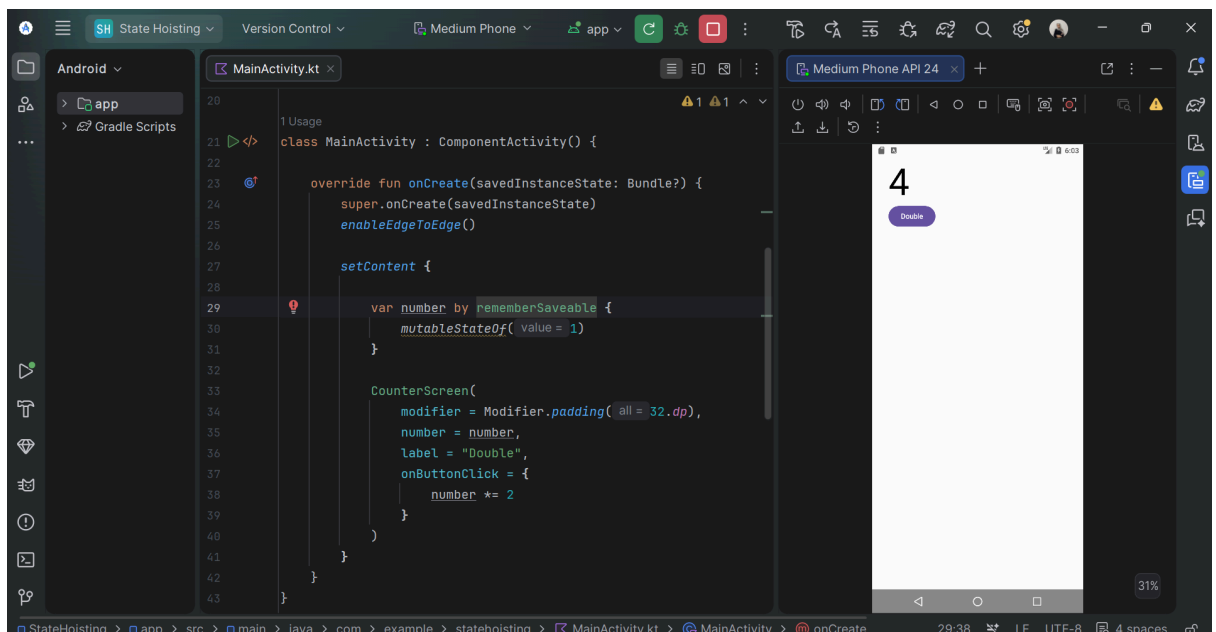
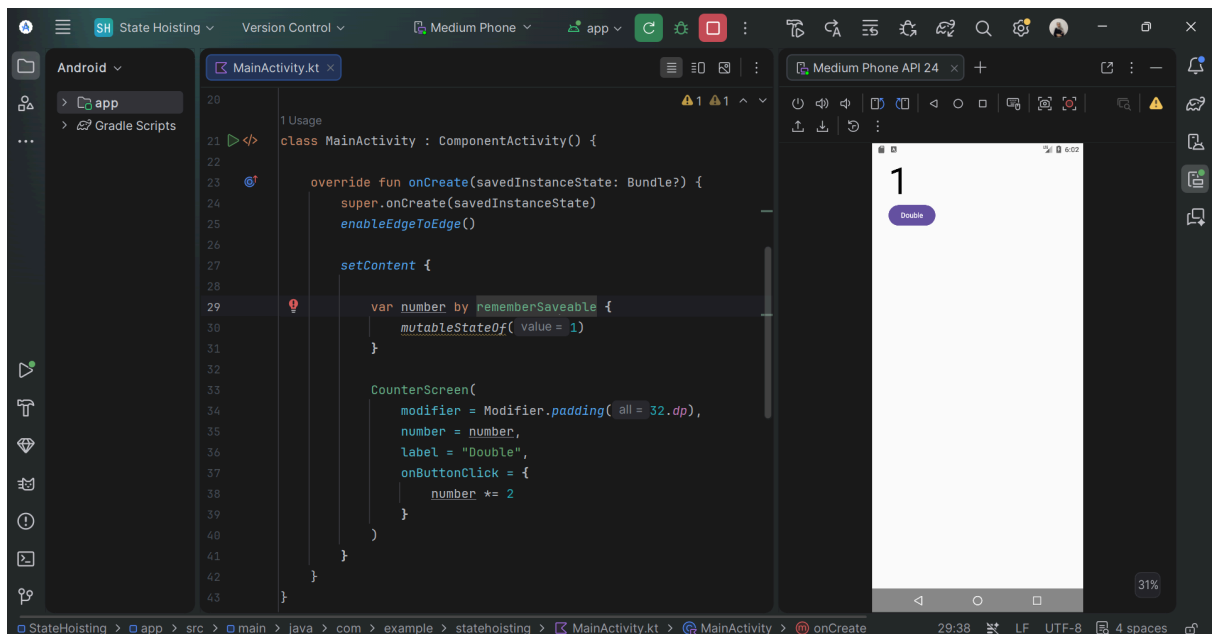
b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan setelah perangkat diputar menjadi landscape adalah 1. Hal ini terjadi karena perubahan orientasi perangkat merupakan configuration change yang menyebabkan Activity dibuat ulang. State number yang sebelumnya disimpan menggunakan remember tidak dapat dipertahankan setelah Activity dibuat ulang, sehingga nilainya kembali ke nilai awal yaitu 1.

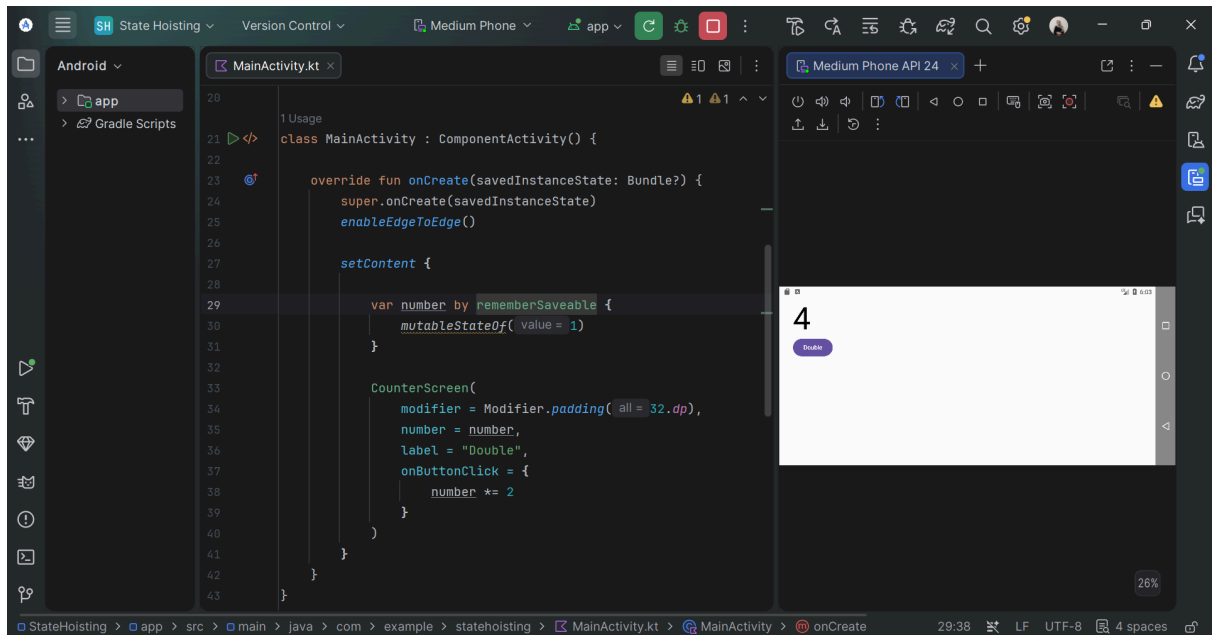
Ubah var number by rememberSaveable { mutableStateOf(1) }



Compile/Run ulang.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan tetap sama seperti sebelum perangkat diputar. Misalnya sebelum rotasi angka yang ditampilkan adalah 8, maka setelah berubah menjadi landscape angka tetap 8. Hal ini terjadi karena `rememberSaveable` dapat menyimpan state sehingga nilai `number` dapat dipulihkan kembali ketika Activity dibuat ulang akibat perubahan konfigurasi.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

`rememberSaveable` berfungsi untuk menyimpan dan memulihkan nilai state ketika terjadi perubahan konfigurasi seperti perubahan orientasi layar. Berbeda dengan `remember`, nilai yang disimpan dengan `rememberSaveable` tidak langsung kembali ke nilai awal ketika Activity dibuat ulang akibat perubahan konfigurasi.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

contoh form pendaftaran atau form pengisian data.

Misalnya pengguna sedang mengisi:

Nama: Andi

Email: andi@gmail.com

Alamat: Jakarta

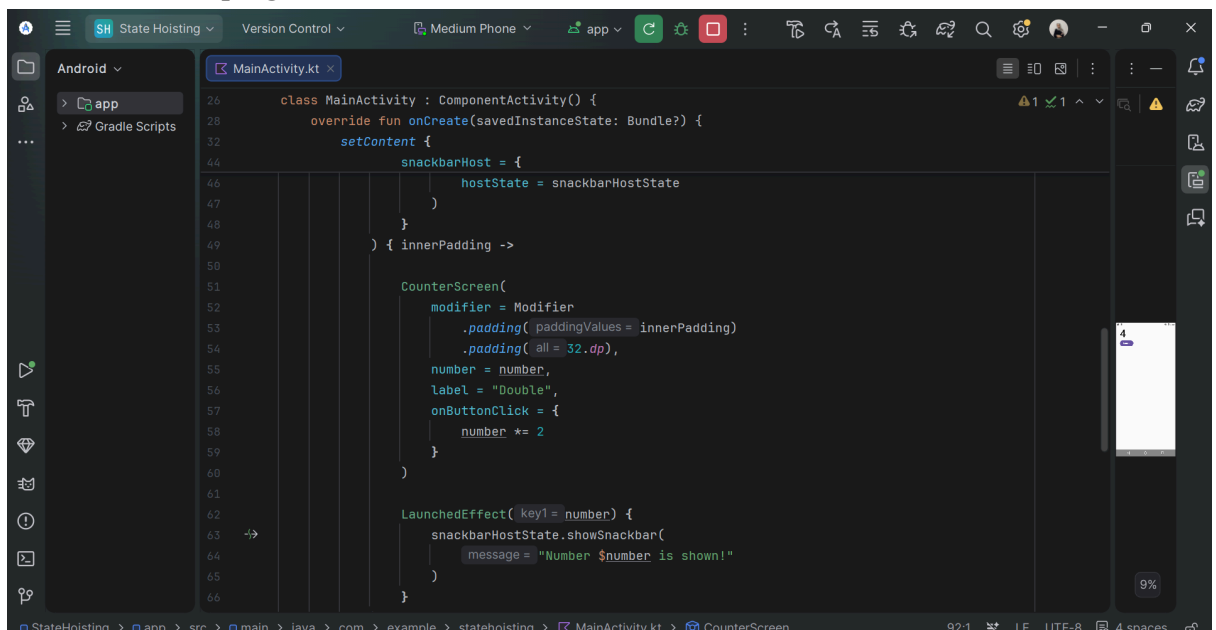
Kemudian pengguna memutar HP dari portrait ke landscape. Kalau menggunakan `remember` saja, nilai input bisa hilang karena Activity dibuat ulang. Dengan `rememberSaveable`, data yang sudah dimasukkan dapat dipulihkan sehingga pengguna tidak perlu mengetik ulang.

Manfaatnya adalah menjaga data sementara pengguna tetap tersedia ketika terjadi perubahan konfigurasi, sehingga pengalaman pengguna menjadi lebih nyaman dan pengguna tidak kehilangan progres yang sedang dikerjakan.

LaunchedEffect

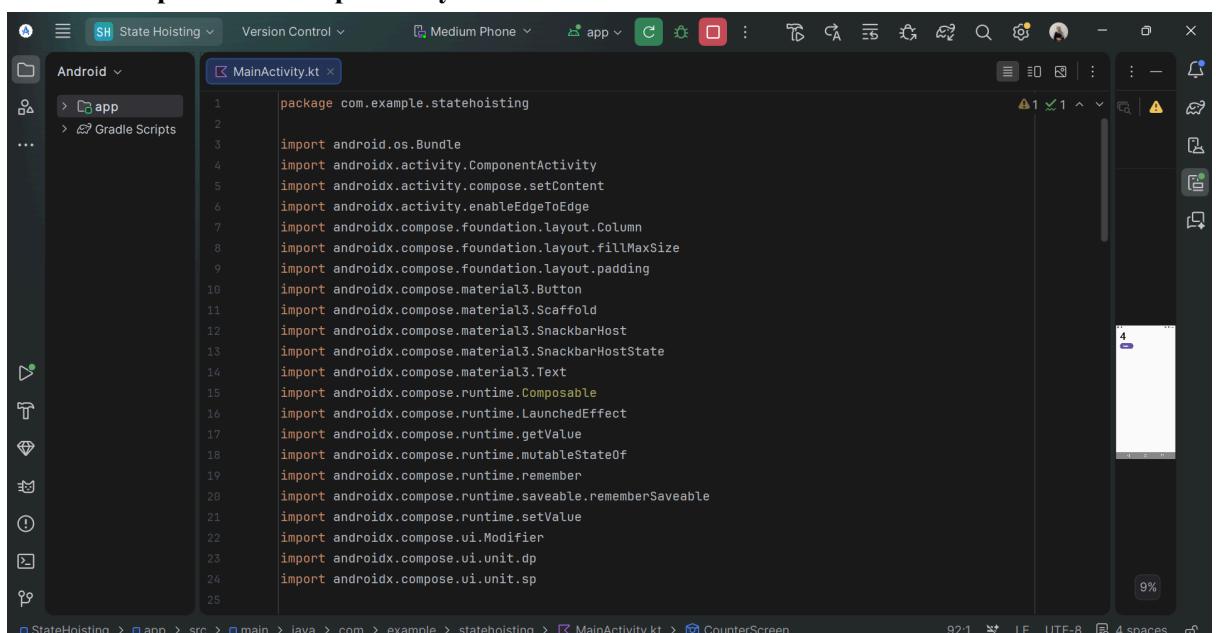
PRA-PERCOBAAN 1

1. Untuk percobaan ini, gunakan Project Android Studio yang sama dengan kode program percobaan Perubahan Konfigurasi setelah menyelesaikan tahap nomor 4 percobaan tersebut.
3. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel number).
4. Mengaitkan composable Snackbar di dalam Scaffold.
5. Menerapkan fitur Side-effect dengan LaunchedEffect. Dengan posisi kursor tetap berada di dalam lambda Scaffold, setelah pemanggilan composable CounterScreen tambahkan kode program side-effect



```
26 class MainActivity : AppCompatActivity() {
27     override fun onCreate(savedInstanceState: Bundle?) {
32         setContent {
44             snackbarHost = {
46                 hostState = snackbarHostState
47             }
48         }
49     } { innerPadding ->
50
51         CounterScreen(
52             modifier = Modifier
53                 .padding(paddingValues = innerPadding)
54                 .padding(all = 32.dp),
55             number = number,
56             label = "Double",
57             onClick = {
58                 number *= 2
59             }
60         )
61
62         LaunchedEffect(key1 = number) {
63             snackbarHostState.showSnackbar(
64                 message = "Number $number is shown!"
65             )
66         }
67     }
68 }
```

6. Lakukan import semua dependency



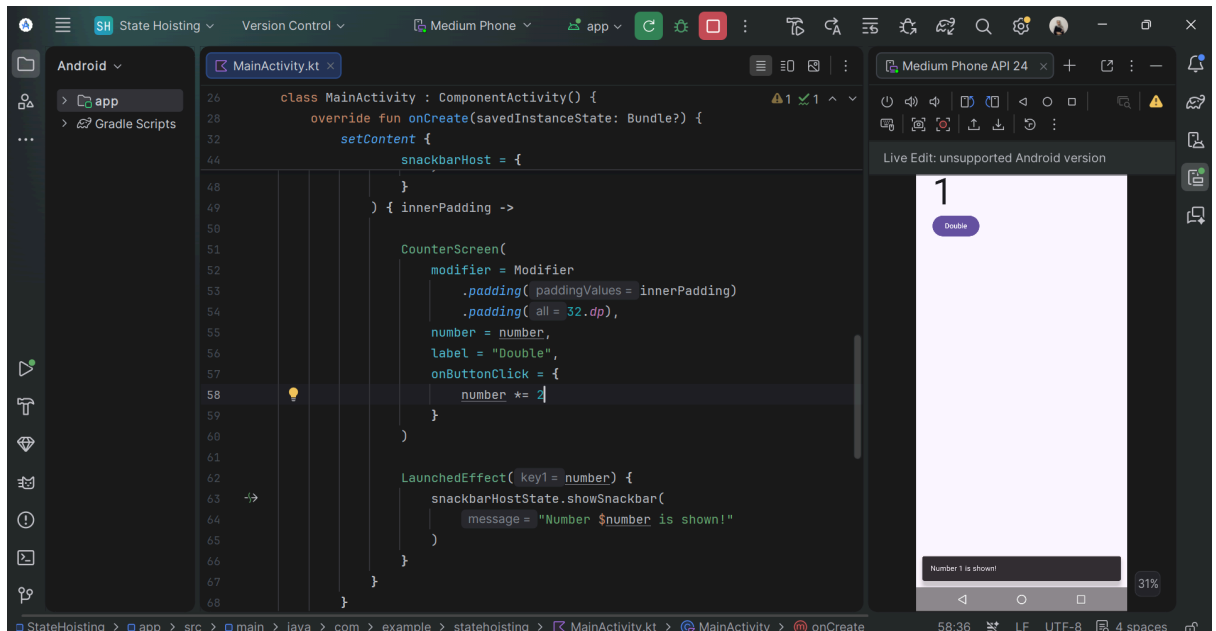
```
1 package com.example.statehoisting
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.Column
8 import androidx.compose.foundation.layout.fillMaxSize
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.material3.Button
11 import androidx.compose.material3.Scaffold
12 import androidx.compose.material3.SnackbarHost
13 import androidx.compose.material3.SnackbarHostState
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.runtime.LaunchedEffect
17 import androidx.compose.runtime.getValue
18 import androidx.compose.runtime.mutableStateOf
19 import androidx.compose.runtime.remember
20 import androidx.compose.runtime.saveable.rememberSaveable
21 import androidx.compose.runtime.setValue
22 import androidx.compose.ui.Modifier
23 import androidx.compose.ui.unit.dp
24 import androidx.compose.ui.unit.sp
25
```

AKTIVITAS PERCOBAAN

1. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?

muncul Snackbar dengan pesan "Number 1 is shown!". Angka 1 berasal dari nilai awal variabel number yang ditentukan pada mutableStateOf(1).



b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

karena LaunchedEffect(number) akan dijalankan ketika Composable pertama kali masuk atau muncul dalam composition. Jadi, meskipun pengguna belum melakukan interaksi apa pun, nilai number sudah tersedia yaitu 1, sehingga LaunchedEffect langsung menjalankan perintah untuk menampilkan Snackbar.

c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

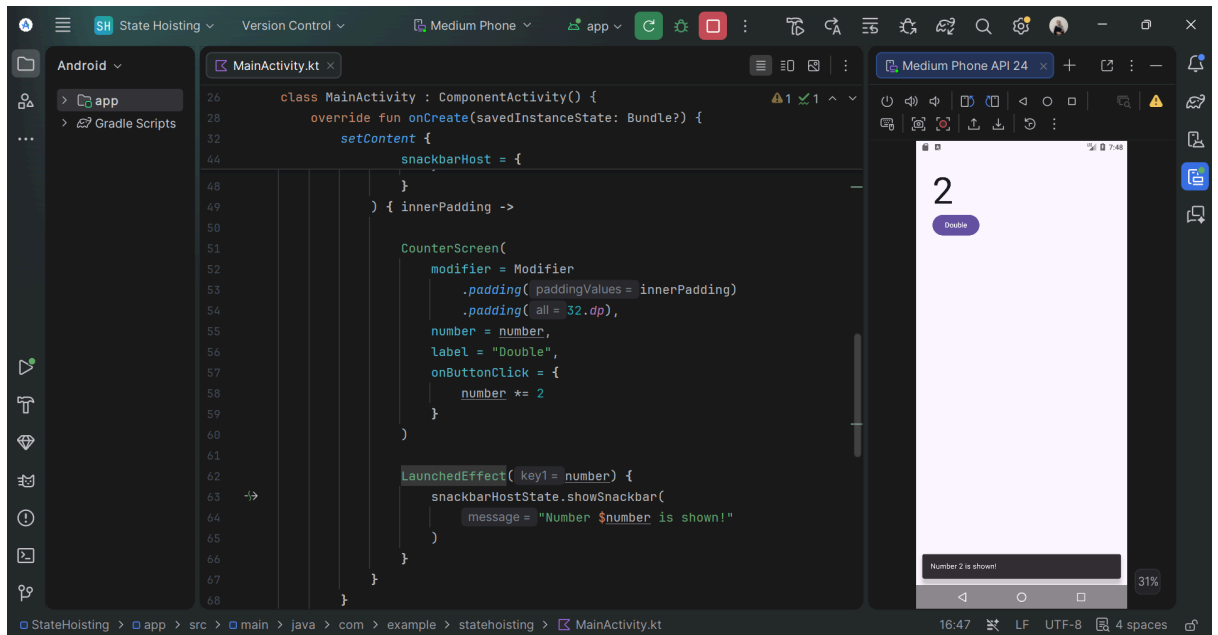
Yang memicu Snackbar ditampilkan adalah LaunchedEffect(number). Bagian yang menjalankan tampilan Snackbar terdapat pada kode:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar(  
        "Number $number is shown!"  
    )  
}
```

number di dalam LaunchedEffect(number) menjadi key atau pemicu efek tersebut. Ketika nilai number berubah, LaunchedEffect akan dijalankan kembali sehingga pesan Snackbar akan menampilkan nilai number yang terbaru.

2. Klik button "Double" satu kali,

a. Apa yang terjadi?



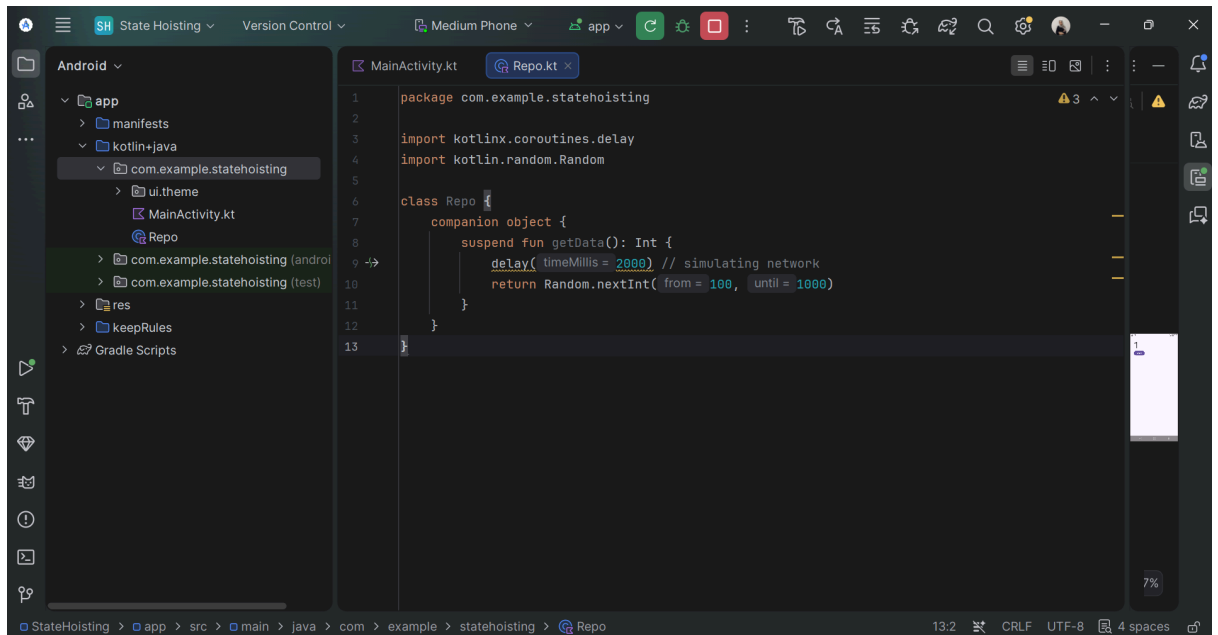
Ketika tombol "Double" diklik satu kali, nilai angka berubah dari 1 menjadi 2. Setelah itu muncul Snackbar dengan pesan "Number 2 is shown!"

b. Mengapa hal tersebut terjadi? Jelaskan!

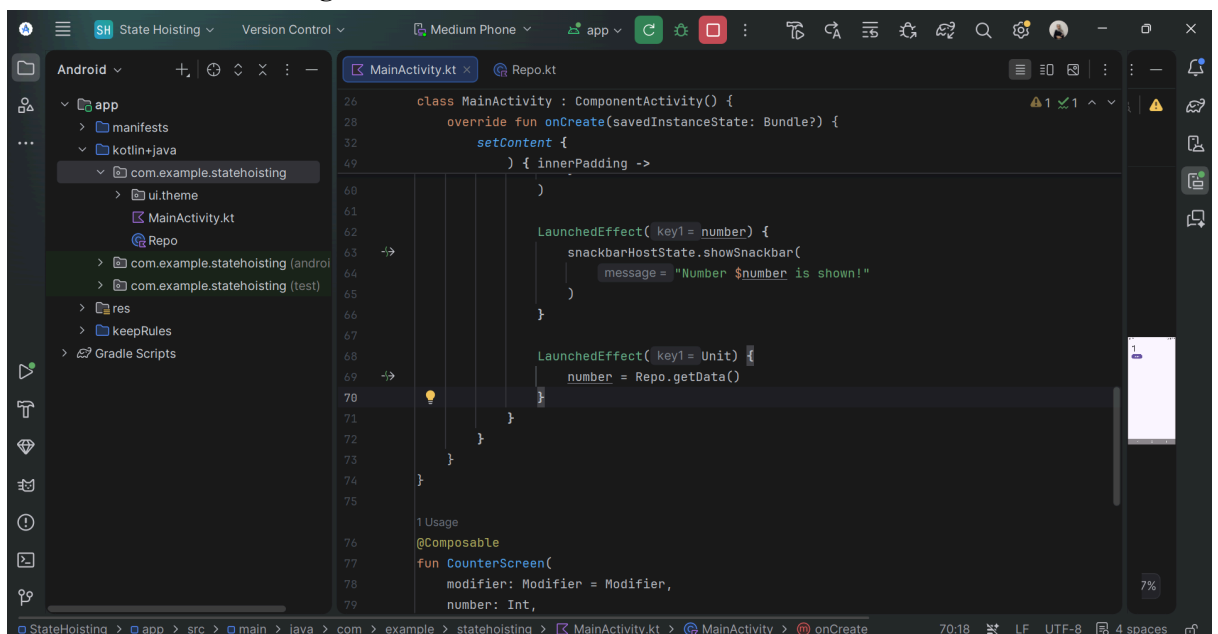
Hal tersebut terjadi karena ketika tombol "Double" diklik, event `onButtonClick` menjalankan `number *= 2`, sehingga nilai `number` berubah dari 1 menjadi 2. Perubahan state tersebut menyebabkan recomposition dan karena `number` digunakan sebagai `key` pada `LaunchedEffect(number)`, efek tersebut dijalankan kembali. Akibatnya, `showSnackbar()` dipanggil dan menampilkan pesan yang sesuai dengan nilai terbaru, yaitu "Number 2 is shown!"

PRA-PERCOBAAN 2

1. Untuk percobaan ini, lanjutkan Project Android Studio yang sama dengan kode program percobaan Side-effect sebelumnya.
2. Di dalam package yang sama dengan class `MainActivity`, buat sebuah class Kotlin baru bernama `Repo` (file: `Repo.kt`) dengan struktur class dan fungsi static `getData()` seperti pada kode program berikut & lakukan import



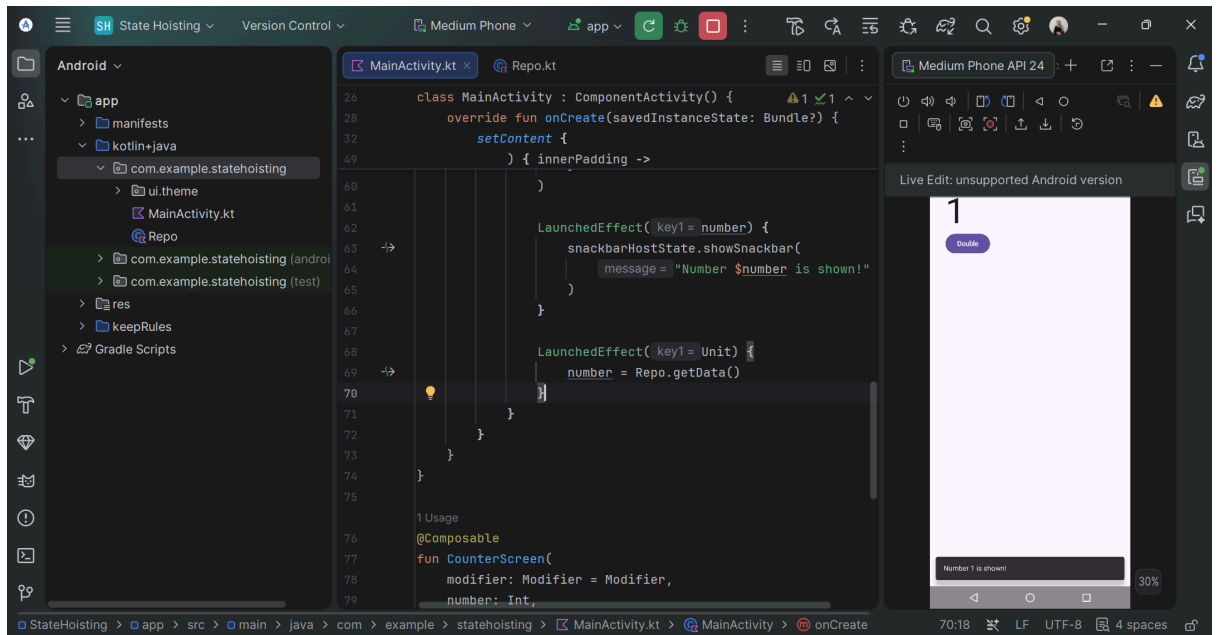
3. Di dalam fungsi onCreate(), setelah LaunchedEffect pada percobaan sebelumnya, tambahkan deklarasi fungsi LaunchedEffect baru berikut



4. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

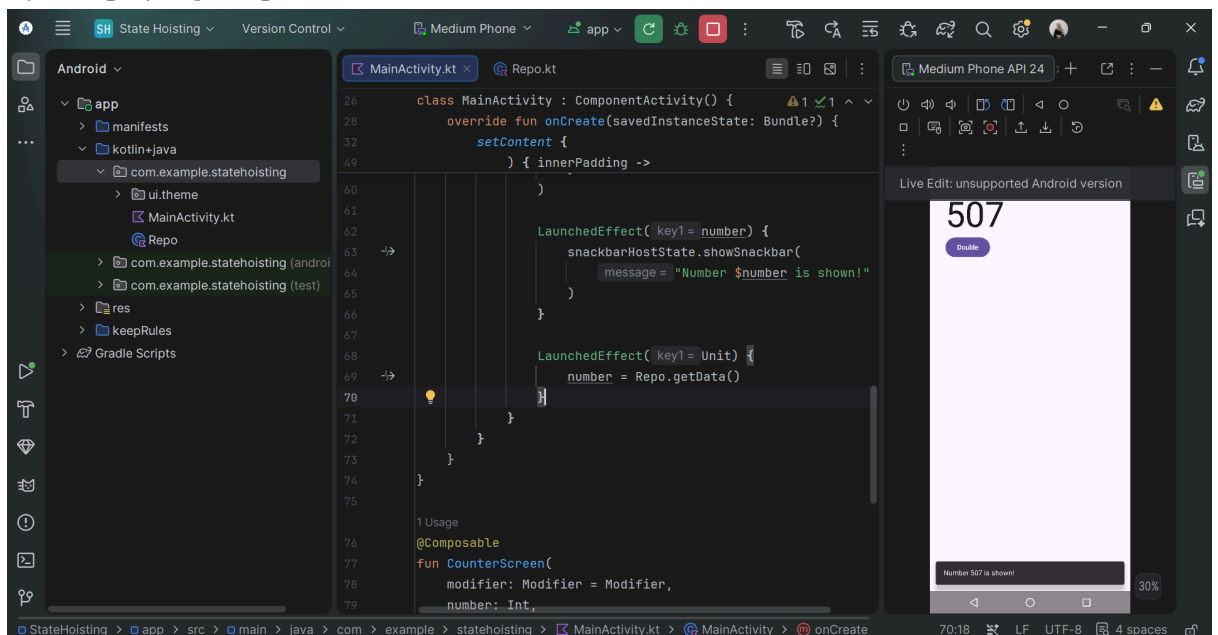
AKTIVITAS PERCOBAAN

1. Jalankan program,
 - c. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



Pada saat pertama kali program dijalankan, angka yang ditampilkan adalah 1, karena nilai awal number masih diatur menjadi 1. Selain itu, Snackbar akan menampilkan pesan "Number 1 is shown!" karena `LaunchedEffect(number)` langsung dijalankan ketika composible pertama kali masuk ke composition.

d. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?



Setelah menunggu sekitar dua detik, angka pada layar akan berubah dari 1 menjadi angka acak antara 100 sampai 999. Hal ini terjadi karena `LaunchedEffect(Unit)` menjalankan `Repo.getData()`, yang memiliki `delay(2000)` sebelum mengembalikan angka random. Setelah nilai tersebut didapatkan, nilai number diubah menjadi angka tersebut. Snackbar kemudian menampilkan pesan sesuai angka baru, misalnya "Number 507 is shown!"

e. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan angka dipicu oleh `LaunchedEffect(Unit)` yang menjalankan `Repo.getData()`. Setelah menunggu dua detik, hasil dari `Repo.getData()` diberikan ke `number` melalui kode `number = Repo.getData()`. Perubahan nilai `number` tersebut kemudian memicu `LaunchedEffect(number)` untuk dijalankan kembali. Karena itu, SnackBar menampilkan pesan baru sesuai dengan angka yang baru diperoleh.

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,

a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

Perbedaannya terletak pada key yang digunakan sebagai pemicu.

`LaunchedEffect(state)` menggunakan nilai `state` sebagai key, sehingga effect akan dijalankan kembali ketika nilai `state` berubah.

Sedangkan `LaunchedEffect(Unit)` menggunakan `Unit` sebagai key yang nilainya tetap, sehingga effect hanya dijalankan ketika pertama kali masuk ke `composition` dan tidak dijalankan kembali hanya karena terjadi perubahan `state` lain.

b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?

`LaunchedEffect(state)` dijalankan ketika `composable` pertama kali masuk ke `composition` dan akan dijalankan kembali setiap kali nilai `state` yang menjadi key berubah. Dalam percobaan ini, ketika `number` berubah dari 1 menjadi angka random, `LaunchedEffect(number)` dijalankan kembali dan menampilkan SnackBar dengan angka terbaru.

Sementara itu, `LaunchedEffect(Unit)` dijalankan ketika `composable` pertama kali masuk ke `composition`. Karena key-nya adalah `Unit` dan tidak berubah, effect ini tidak dijalankan kembali akibat perubahan `number`. Dalam percobaan ini, effect tersebut digunakan untuk mengambil data dari `Repo` setelah menunggu dua detik.